

Screen Sketches

2_jubair_1

Muhammad Blal, Ayaan Syed, Aniroop Naladala, Aparneesh Patil

Actors

1. Authenticated User (real person using the app)

- Can play one of two roles inside a trip:
 - Trip Member (role): join/leave by code/link; see itinerary/packing/weather; chat & @mention; share/stop live location; add expenses & see balances; write journal entries; get notifications.
 - Trip Owner (role): everything a Member can do plus create/edit/archive trips; invite/kick and assign roles; edit itinerary/packing templates; set budget categories; export summaries.

2. Admin Moderator (privileged user at platform level)

- Remove abusive content or spam; suspend/reinstate users; handle flags; feature trips; manage roles.

3. External APIs (system actor)

- Weather, maps/geocoding, currency rates, push notifications, and image storage that our backend calls.

Note: Member and Owner are roles of the same user actor. We list both to show what each can do.

Non-Functional Requirements

1. Correctness & Reliability (Data Safety)

- Expense/transfer and membership updates happen in one MariaDB transaction (no half-saved data or double charges).
- The app shows “success” only after the server saves the data.
- (Simple correctness check) For each trip, expense splits should balance out across members (sum of balances $\sim$ \$0.00, allowing normal cent-rounding).

2. Performance (how fast it should feel)

- For most requests ($\sim$ 95%), reads finish in $\sim$ 0.3 seconds and writes in $\sim$ 0.6 seconds.
- Measured on the server: from when our Spring Boot controller starts handling a request to when it returns a response (not counting phone rendering or Wi-Fi/cell speed).
- Chat: sender taps “Send” $\rightarrow$ others see it in $\sim$ 1 second.
- Media uploads: server acknowledges in $\sim$ 2 seconds on campus Wi-Fi (thumbnails can finish later).

3. Availability

- Backend up 99% of the month on the ISU server (planned maintenance excluded).

4. Security & Privacy

- Auth via server-managed session or opaque API token stored/validated in MariaDB (no JWT).
- Passwords hashed with BCrypt; HTTPS required.
- Location sharing is opt-in and can be paused anytime.

5. Scalability

- Handles $\sim$ 100 signed-in users across the app at once.
- Handles $\sim$ 10 people in the same trip actively chatting/editing without obvious slowdown.
- Prevent accidental overwrites on concurrent edits (simple version checks or blocking).

6. Usability

- Critical actions (send message, add expense, mark item packed) are reachable in ≤ 3 taps from the trip dashboard.
- Show a visible loading indicator for operations taking >150 ms; show a clear error with Retry on failures.
- Back navigation returns to the previous app screen (no dead-ends).

7. Maintainability

- Spring Boot with controller/service/repository layers; basic OpenAPI docs.
- ~70% unit test coverage for the service layer.
- CI runs tests on every merge request.

8. Portability / Where it runs

- Android app ↔ Spring Boot REST/JSON on the 2_jubair_1 ISU remote server.
- Database: MariaDB for all tables/info

9. Observability

- Logs include a request ID; the same ID appears in error responses.
- Basic counters for trips created, chat messages, and expense operations.

Tables and Fields

- **Users:** Tracks information related to a user's account
 - User Id - Primary Key
 - Username
 - Email
 - Hashed Password
 - First Name
 - Last Name
 - Avatar URL
 - Role (ADMIN / TRIP_OWNER / TRIP_MEMBER)
 - Active - used for soft delete of account
 - Created At
 - Updated At
 - Last Login Time
- **User_Settings:** per-user preferences (one-to-one with Users)
 - User_Settings Id - Primary Key
 - User Id - foreign key to Users (unique; one row per user)
 - Privacy Settings (e.g., default location sharing, profile visibility)
 - Notification Preferences (e.g., push on/off, quiet hours)
 - Time Zone
 - Updated At
- **Trips:** travel groups a user can own and others can join
 - Trip Id - Primary Key
 - Owner Id - foreign key to Users
 - Trip Name
 - Destination
 - Start Date
 - End Date
 - Created At
 - Updated At
 - Archived
- **Trip_Members:** membership of users in trips (many-to-many with per-trip role)
 - Trip_Member Id - Primary Key
 - Trip Id - foreign key to Trips
 - User Id - foreign key to Users
 - Role in Trip (OWNER / MEMBER)
 - Joined At
 - Can Share Location

- Unique: (Trip Id, User Id)
- **Events:** per-user calendar items (reminders, itinerary blocks)
 - Event Id - Primary Key
 - User Id - foreign key to Users
 - Title
 - Description
 - Start At
 - End At
 - Location
 - All Day
 - Created At
 - Updated At
- **Tasks:** personal to-dos
 - Task Id - Primary Key
 - User Id - foreign key to Users
 - Title
 - Description
 - Status (e.g., TODO / DONE)
 - Due Date
 - Created At
 - Updated At
- **Labels (combined with Task_Labels):** attaches labels directly to tasks
 - Label Id - Primary Key
 - Task Id - foreign key to Tasks
 - User Id - foreign key to Users (owner of the label entry)
 - Name
 - Color (e.g., "#1976D2")
 - Created At
 - Unique: (Task Id, Name) - prevents duplicate label names on the same task
- **Friendships:** contacts for "add/suggest friends"
 - Friendship Id - Primary Key
 - User Id - foreign key to Users
 - Friend User Id - foreign key to Users
 - Status (PENDING / ACCEPTED / BLOCKED)
 - Created At
 - Unique: (User Id, Friend User Id)

- **Direct_Messages:** one-to-one messages with friends
 - Direct_Message Id - Primary Key
 - Sender Id - foreign key to Users
 - Recipient Id - foreign key to Users
 - Content
 - Parent DM Id - foreign key to Direct_Messages (reply/forward)
 - Sent At
 - Saved At
 - Is Read

- **Messages:** group chat messages inside a trip
 - Message Id - Primary Key
 - Trip Id - foreign key to Trips
 - Sender Id - foreign key to Users
 - Content
 - Parent Message Id - foreign key to Messages (reply/forward)
 - Sent At - time the user sent it
 - Saved At - time the server stored it
 - Is Edited

- **Message_Reactions:** emoji reactions on messages
 - Reaction Id - Primary Key
 - Message Id - foreign key to Messages
 - User Id - foreign key to Users
 - Emoji
 - Reacted At
 - Unique: (Message Id, User Id, Emoji) - one reaction type per user per message

- **Notifications:** invites, approvals, task/packing reminders
 - Notification Id - Primary Key
 - User Id - foreign key to Users (recipient)
 - Trip Id - foreign key to Trips (optional context)
 - Event Id - foreign key to Events (optional)
 - Task Id - foreign key to Tasks (optional)
 - Type (INVITE / APPROVAL / TASK / REMINDER)
 - Message
 - Is Read (seen/unseen toggle)
 - Created At

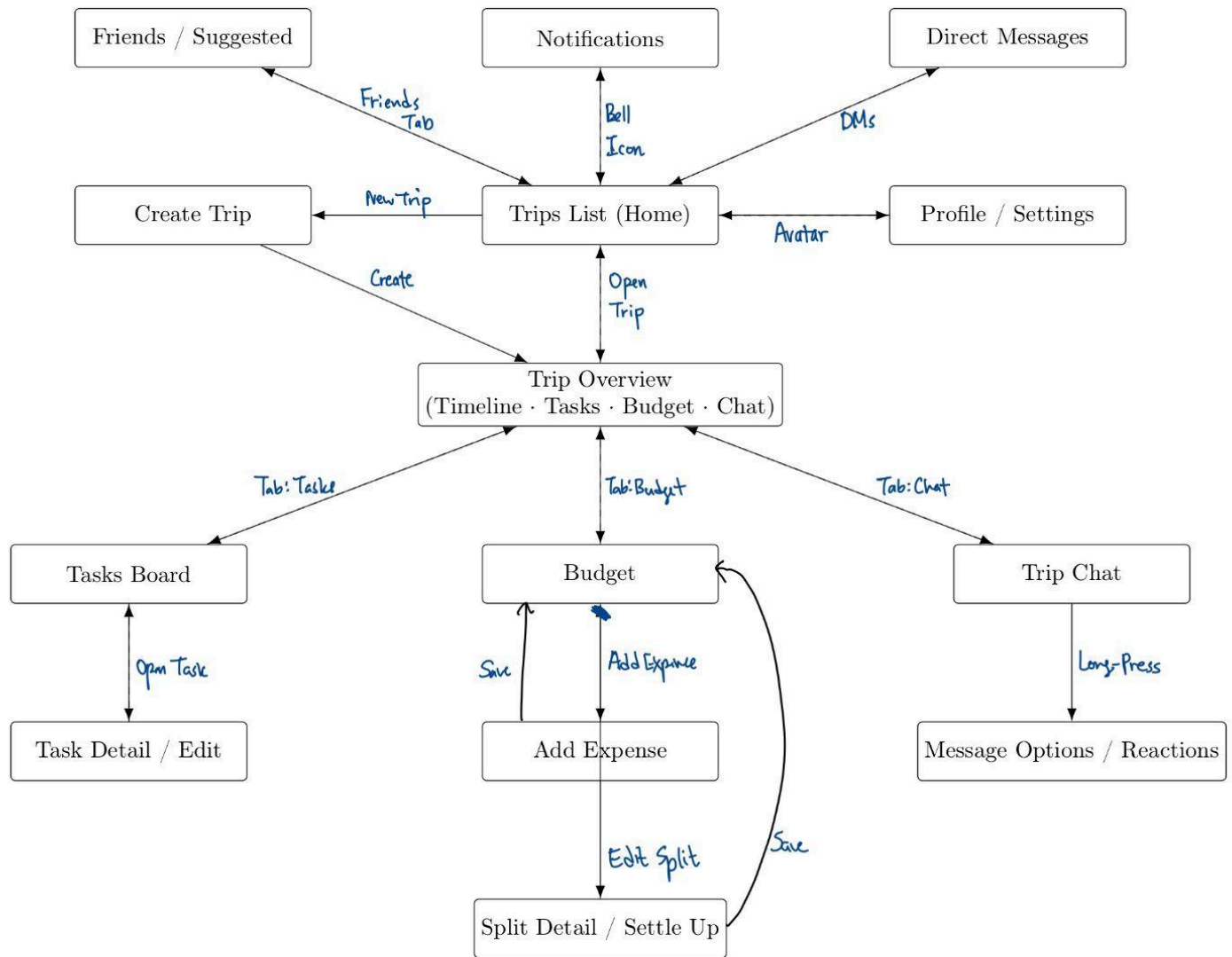
- **Budget:** trip purchase records
 - Budget Id - Primary Key
 - Trip Id - foreign key to Trips
 - Paid By - foreign key to Users
 - Amount
 - Description
 - Created At

- **Budget_Splits:** who owes what for a given budget entry
 - Split Id - Primary Key
 - Budget Id - foreign key to Budget
 - User Id - foreign key to Users
 - Owed Amount

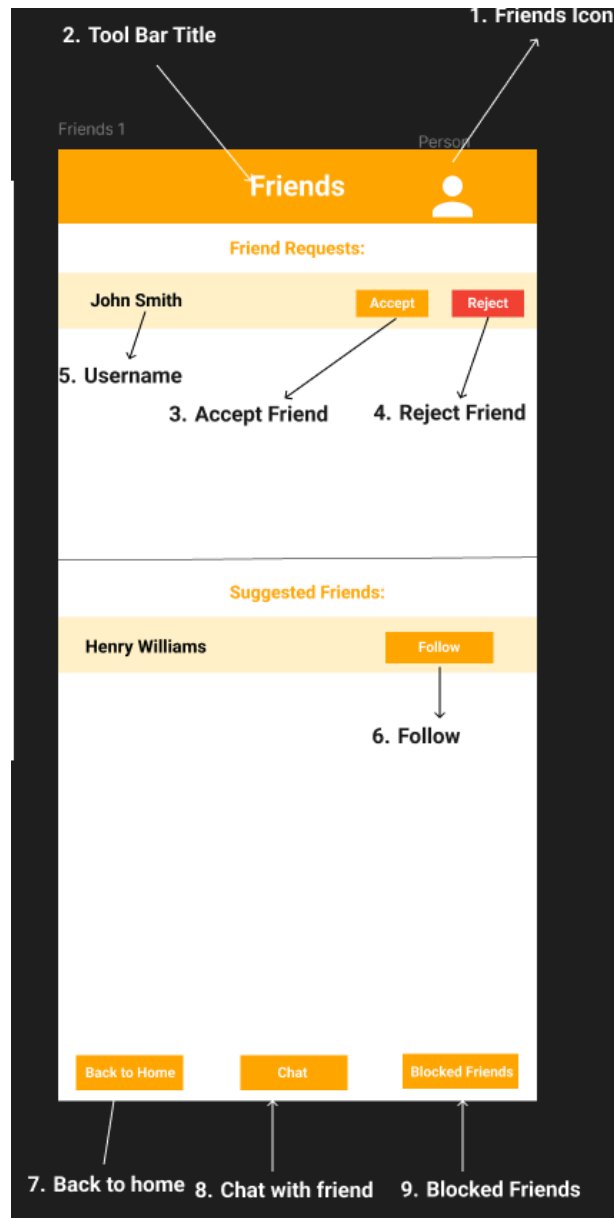
- **Packing_Items:** if we later separate packing from Tasks
 - Packing_Item Id - Primary Key
 - Trip Id - foreign key to Trips
 - Added By - foreign key to Users
 - Name
 - Checked (true/false)
 - Created At

- **Travel_Suggestions:** dynamic trip ideas by month/dates
 - Suggestion Id - Primary Key
 - User Id - foreign key to Users (who requested it)
 - Month / Date Range
 - Place / Region
 - Reason / Notes
 - Created At

Screen Flow Diagram



Screen Sketch 1 (Anirop Naladala):

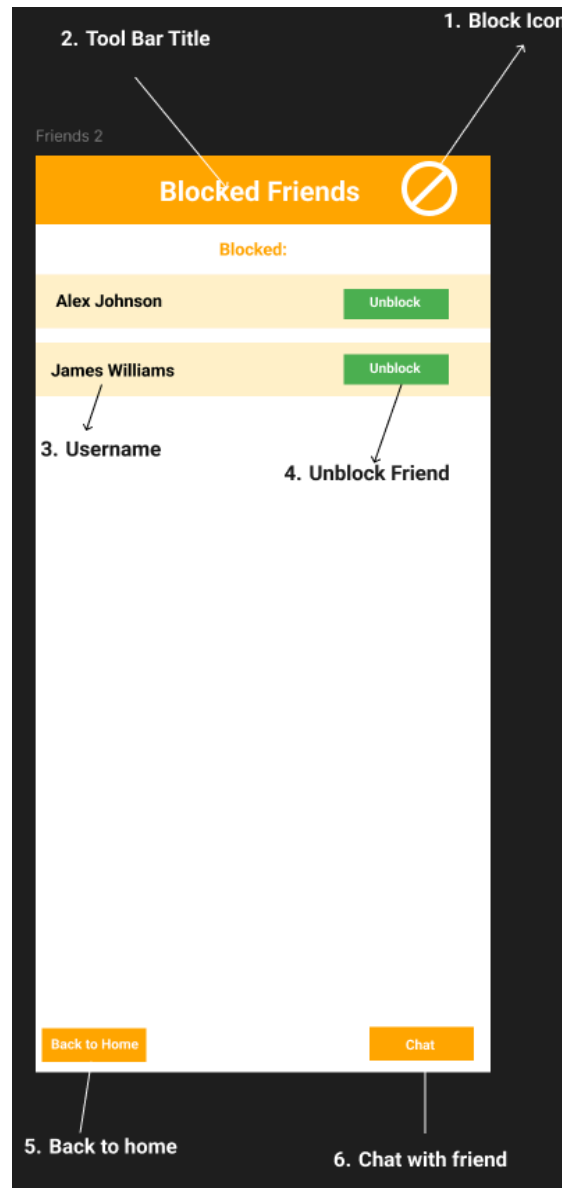


The **Friends screen** in QuestBuddy acts as the main hub for managing and growing a user's social network. It lets users view friend requests, find suggested friends, and navigate related features.

At the top, the **(1) friends icon** and **(2) toolbar title** clearly indicate the section while maintaining consistent styling across the app.

The Friend Requests area shows each sender's **(5) username** with options to **(3) Accept Friend** or **(4) Reject Friend**, making friend management simple. Below it, the Suggested Friends section lists new connections with **(6) Follow** buttons to promote engagement.

Finally, the bottom navigation includes **(7) Back to Home**, **(8) Chat with Friend**, and **(9) Blocked Friends**, providing easy access to key social features.

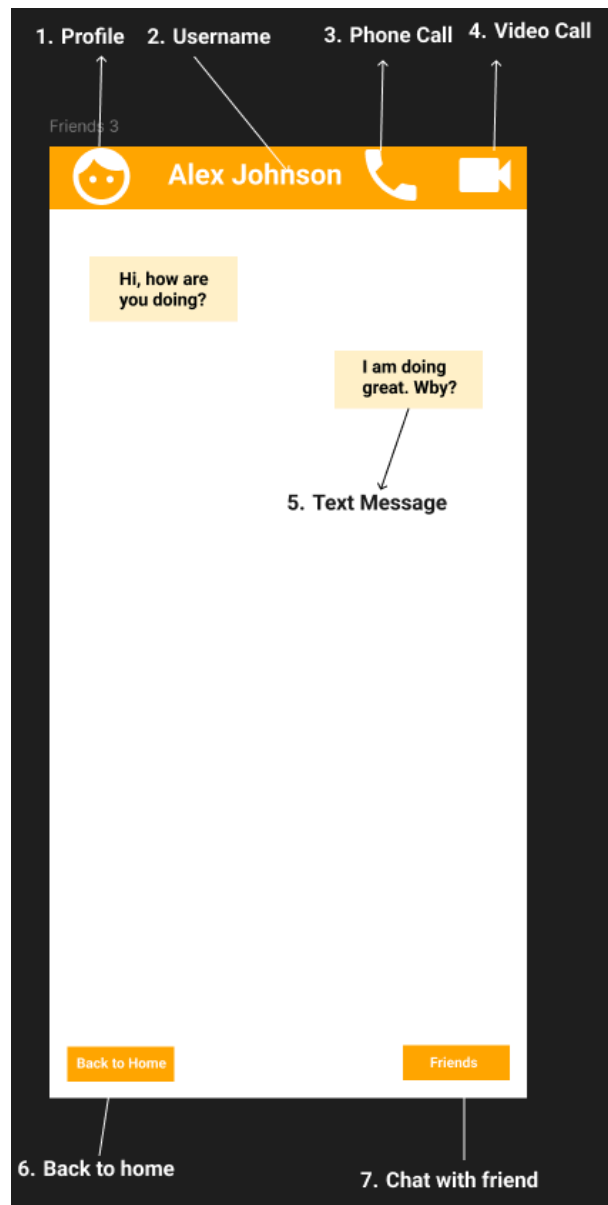


The **Blocked Friends screen** in QuestBuddy provides users with an easy way to manage their blocked connections. It offers a clean layout to view and unblock friends while maintaining access to essential navigation features.

At the top, the **(1) block icon** and **(2) toolbar title** clearly indicate the screen's purpose and maintain consistency with the app's overall design.

In the main section, each blocked user's **(3) username** is listed alongside an **(4) Unblock Friend** button, allowing quick management of blocked connections.

At the bottom, navigation buttons — **(5) Back to Home** and **(6) Chat with Friend** — provide seamless movement between key areas of the app.



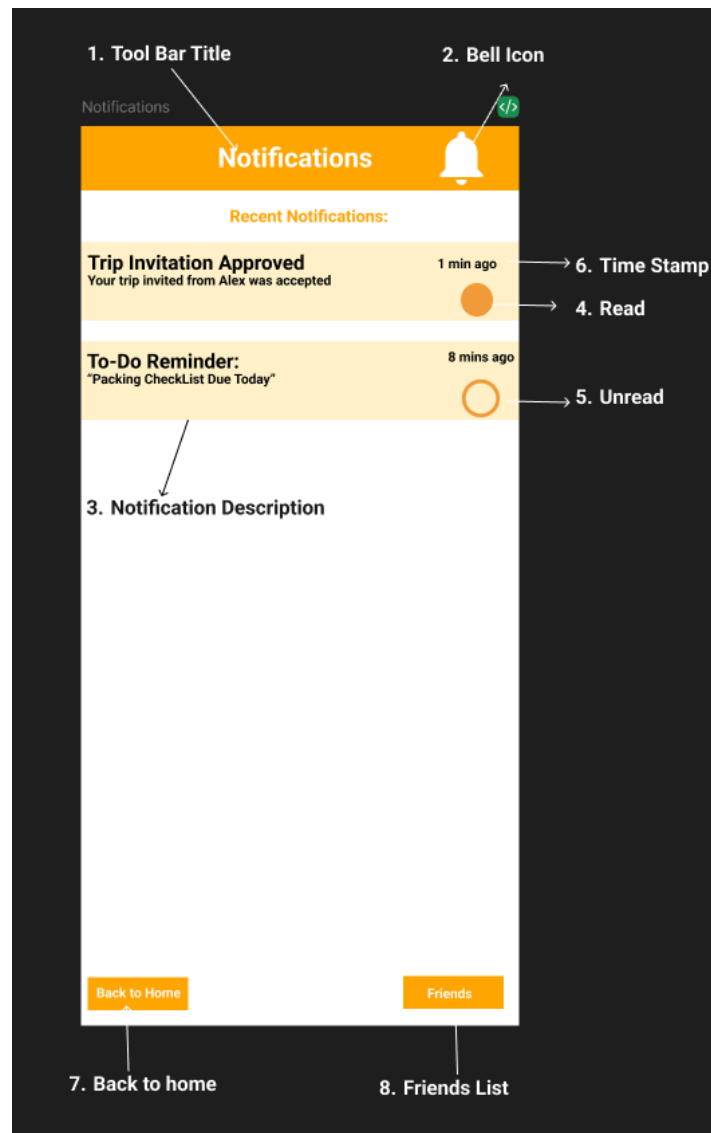
The **Chat screen** in QuestBuddy allows users to communicate directly with their friends through text, voice, or video. It provides a simple and intuitive layout that supports seamless, real-time interaction.

At the top, the **(1) profile** image and **(2) username** identify the friend being chatted with, while the **(3) phone call** and **(4) video call** icons give users quick access to alternative communication methods.

In the main area, the **(5) text messages** appear in a conversational format, showing both sent and received messages clearly for easy reading.

At the bottom, navigation buttons — **(6) Back to Home** and **(7) Chat with Friend** — allow smooth transitions between the chat and other sections of the app.

Screen Sketch 2 (Aniroop Naladala):



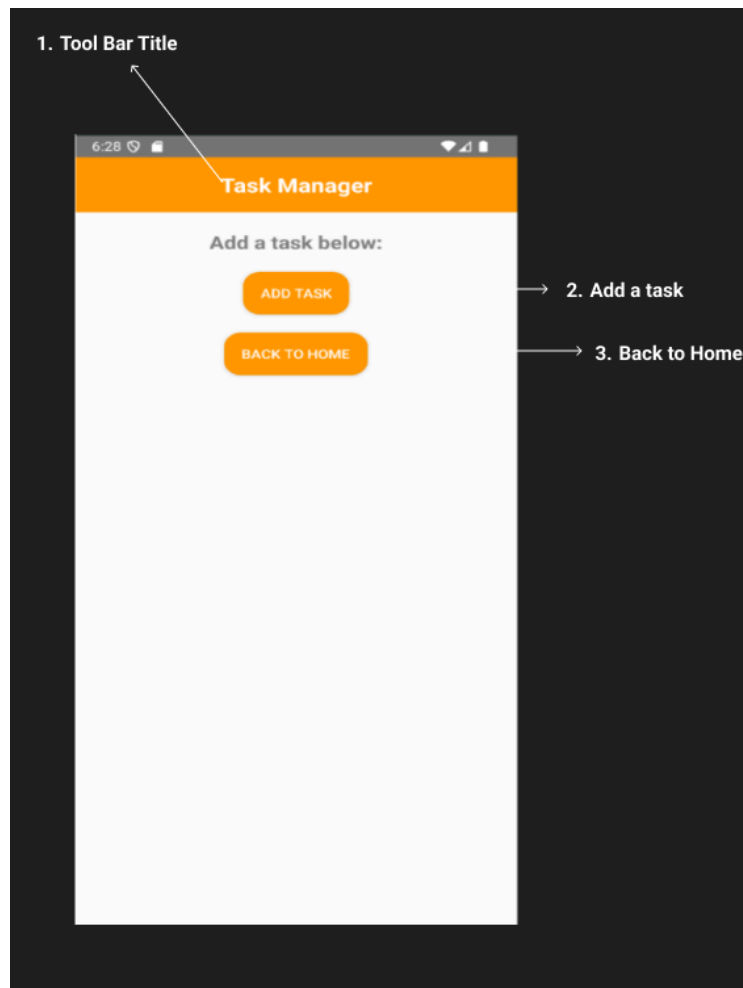
The **Notifications** screen in QuestBuddy keeps users informed about recent updates and activities within the app. It offers a clear, structured layout for viewing both read and unread notifications.

At the top, the (1) **tool bar title** and (2) **bell icon** identify the section, maintaining consistency with other screens in QuestBuddy's interface.

Each notification includes a (3) **notification description** and a (6) **time stamp**, giving users context and timing for each update. Visual indicators mark whether a notification is (4) **read** or (5) **unread**, ensuring quick recognition of new alerts.

At the bottom, the navigation buttons — **(7) Back to Home** and **(8) Friends List** — allow users to easily move between the main dashboard and their social connections.

Screen Sketch 3 (Before and After Adding a Task): Muhammad Blal

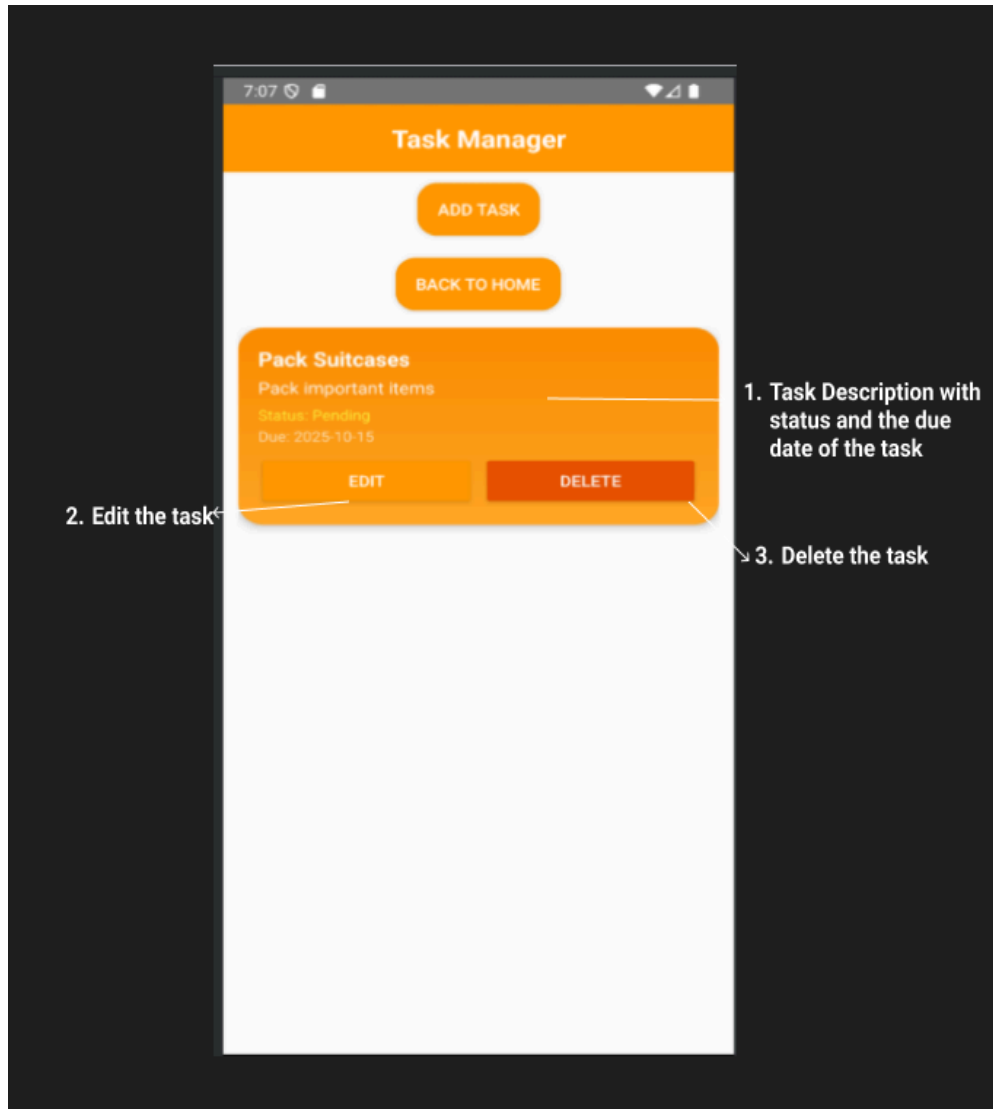


The **Task Manager screen** in QuestBuddy allows users to create and manage their personal tasks efficiently. It maintains a clean and straightforward design for quick navigation and usability.

At the top, the **(1) tool bar title** clearly labels the section, ensuring users know they are within the task management area.

In the main section, users can tap the **(2) Add a Task** button to create new tasks, streamlining productivity and organization. The “Add a task below” prompt automatically **disappears when at least one task is added** and **reappears when a task is deleted** or when there are **no tasks remaining**, providing a dynamic and clutter-free interface.

At the bottom, the **(3) Back to Home** button provides a simple way to return to the main screen, keeping navigation consistent across the app.



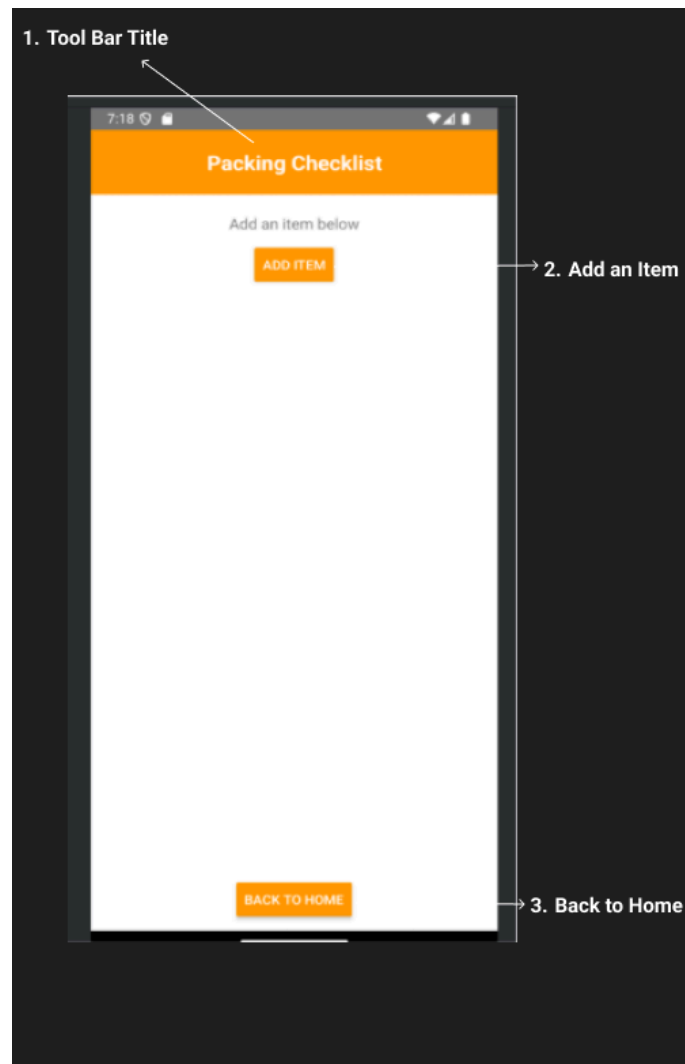
The **Task Manager screen** in QuestBuddy helps users view, update, and organize their active tasks efficiently. It combines clarity and functionality to keep task management simple and accessible.

At the top, the toolbar and task creation buttons remain consistent with the app's design, allowing users to easily add new tasks or return home.

Each task card displays a **(1) task description** that includes the task's details, current **status**, and **due date**, giving users an at-a-glance overview of their progress.

Below each task, users can choose to **(2) edit the task** to make changes or **(3) delete the task** to remove it entirely, giving full control over their task list.

Screen Sketch 4 (Muhammad Blal):

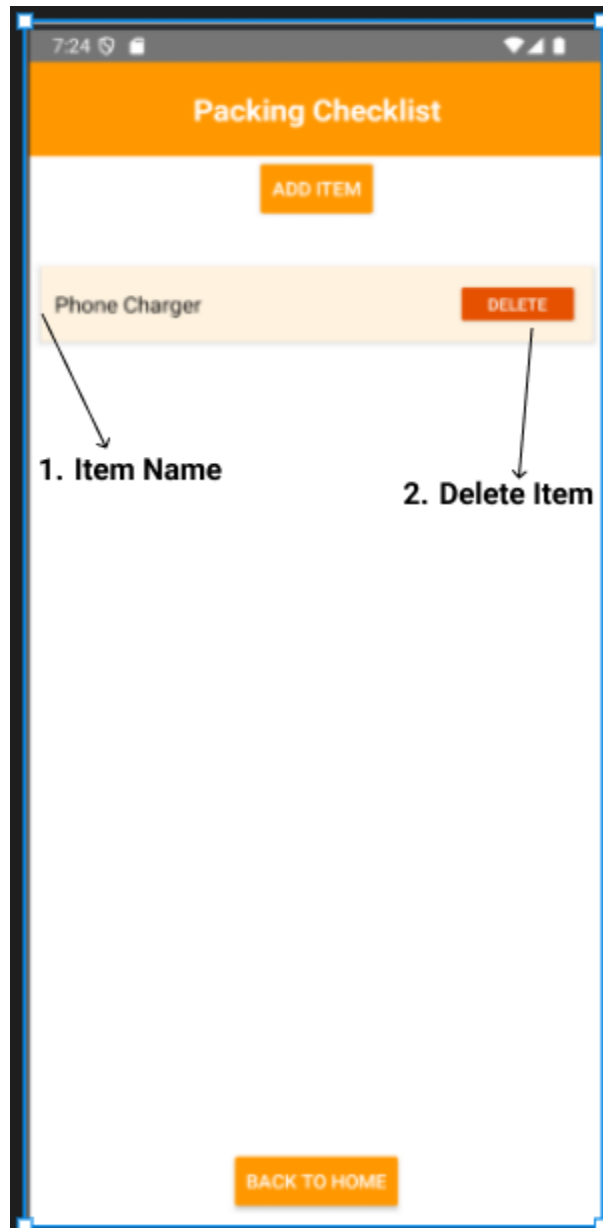


The **Packing Checklist screen** in QuestBuddy helps users organize and manage items they need to pack efficiently. Its simple and intuitive layout ensures a smooth user experience focused on clarity and ease of use.

At the top, the **(1) tool bar title** clearly identifies the section, ensuring users know they are viewing their packing checklist.

In the main section, the **(2) Add an Item** button allows users to include new packing items quickly. The “Add an item below” prompt automatically **disappears once an item is added** and **reappears when all items are deleted** or when **no items remain**, maintaining a clean and responsive interface.

At the bottom, the **(3) Back to Home** button enables users to easily return to the main screen, keeping navigation consistent across the app.



The **Packing Checklist screen** in QuestBuddy allows users to view and manage their packing items in an organized manner. It ensures a clear and efficient experience for keeping track of essential belongings.

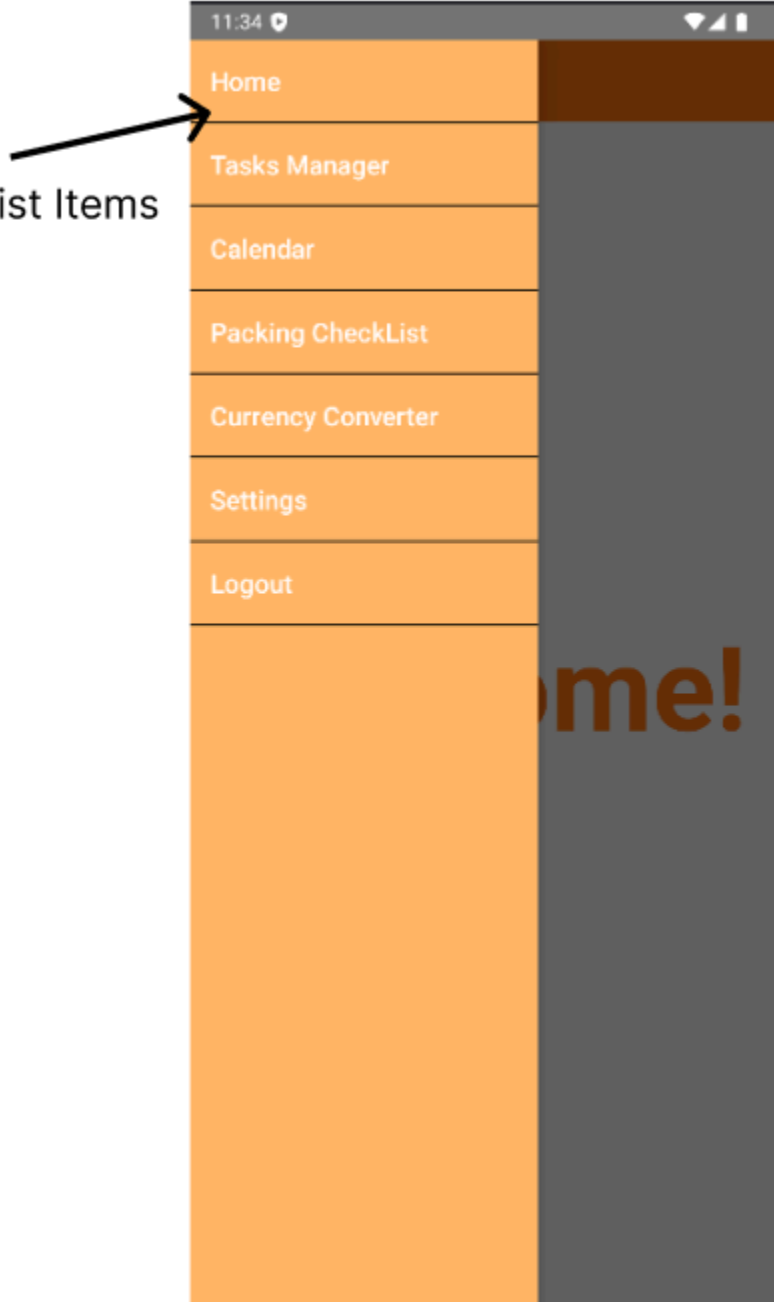
Each entry displays the **(1) item name**, helping users identify what has been added to their list. To maintain flexibility, users can remove unnecessary entries using the **(2) Delete Item** button, which instantly updates the checklist.

When all items are deleted or none remain, the “Add an item below” prompt reappears, signaling that the list is empty and ready for new entries — keeping the interface dynamic and user-friendly.

Screen Sketch 5 (Aparneesh Patil):



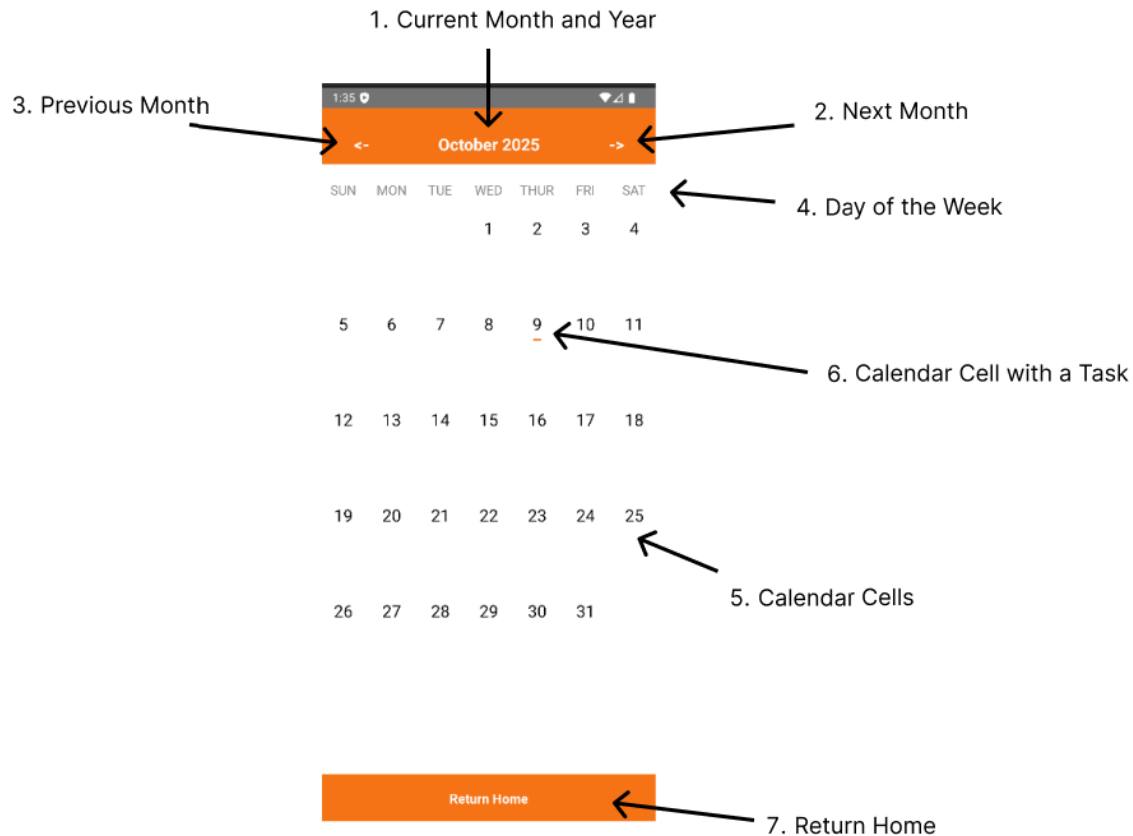
6. Drawer List Items



The QuestBuddy home screen serves as the primary entry point, providing access to key features and personalized content. It greets the user with a **(5) prominent welcome title** and the signed-in user's **(4) username**, ensuring quick orientation while keeping global navigation immediately accessible. The layout emphasizes clarity, responsiveness, and readability across devices, with sensible loading and empty states to maintain a smooth experience.

At the top, the **(1) toolbar** establishes page identity and navigation. It includes a **(2) toolbar title** and a **(3) hamburger icon**; tapping the hamburger icon opens the navigation drawer to reveal the **(6) drawer list items**, each routing to its respective screen.

Screen Sketch 6 (Aparneesh Patil):

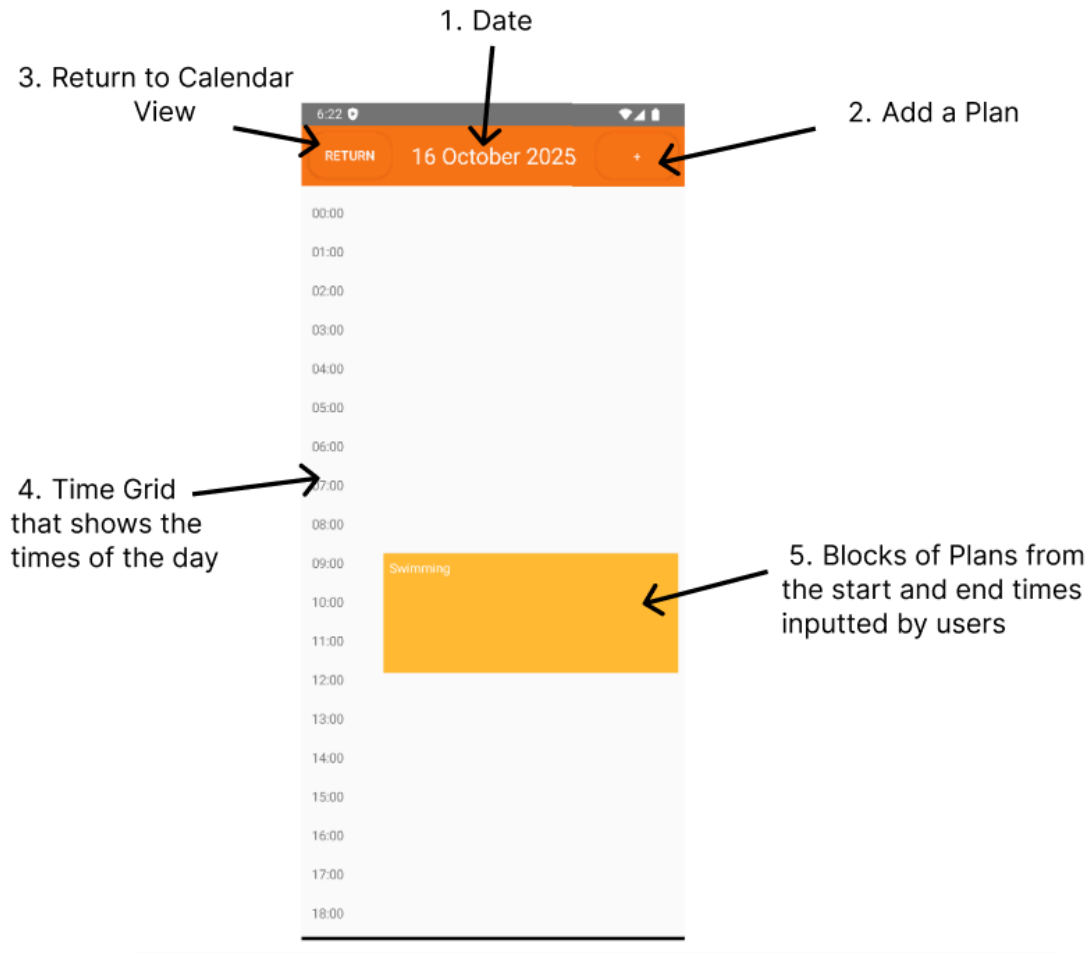


The calendar screen enables users to browse dates, select a specific day, and add tasks directly to it. Month-to-month navigation is smooth and efficient—users can use the dedicated controls (arrows) for rapid movement across the calendar while maintaining clear context.

At the top, the toolbar displays the **(1) current month and year**, with two buttons: one for the **(2) next month** and another for the **(3) previous month**. Below the toolbar, **(4) seven horizontally aligned TextViews** label the days of the week, and a RecyclerView renders the **(5) calendar cells**, recycling views so dates align precisely under their corresponding weekdays. When a date contains at least one task, **(6) a small orange indicator** appears beneath that day within its cell for quick visual scanning.

For broader app navigation, the **(7) return-home button** at the bottom takes users back to the home screen, making it easy to pivot to other features without losing progress.

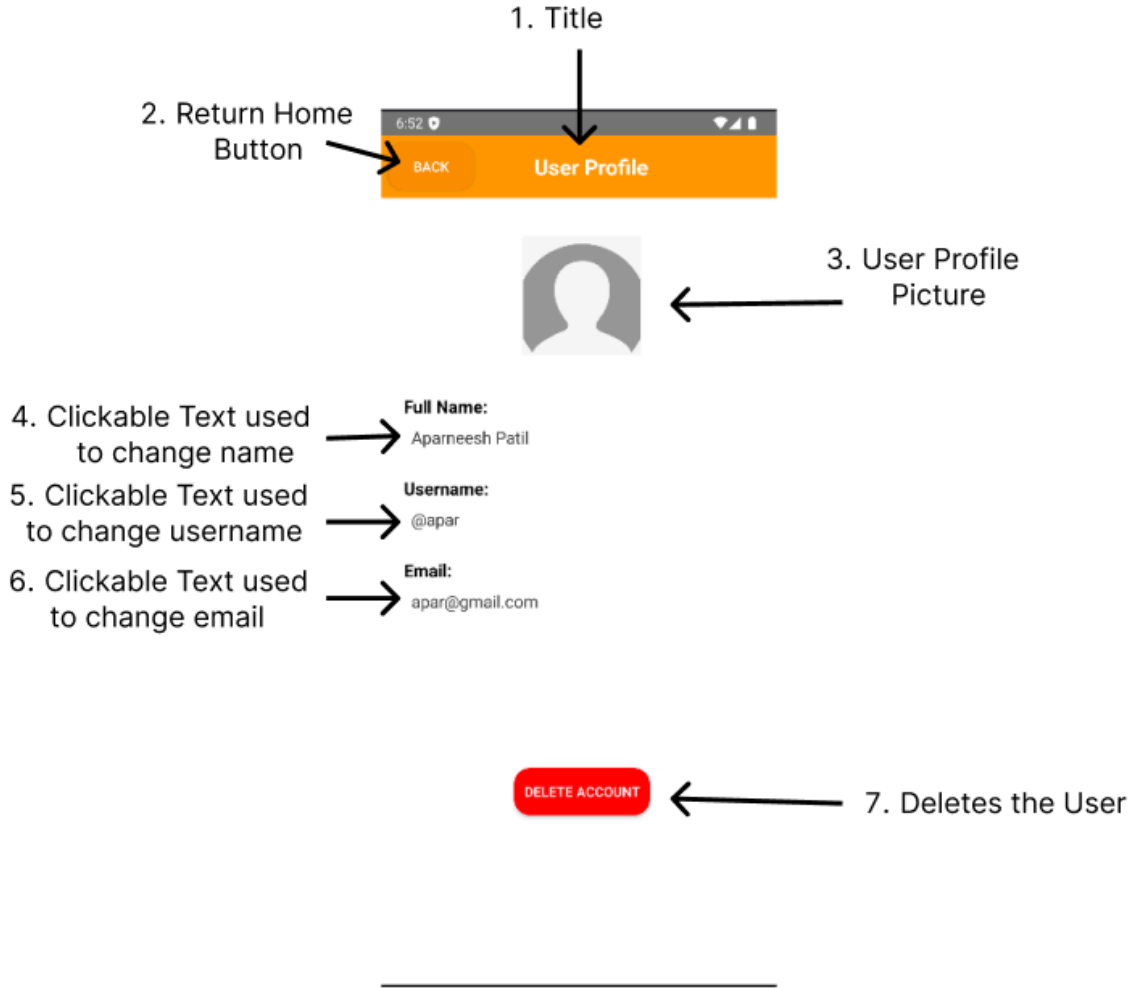
Screen Sketch 7 (Ayaan Syed):



The Day View page presents users' schedules in a block-based timeline for a chosen time range, making it easy to scan what's planned at a glance. Users can add tasks by specifying start and end times; each entry is placed on the timeline accordingly, supporting clear planning and quick conflict detection.

At the top, the toolbar clearly shows the **(1) selected date** so users always know which day is being edited. It also includes a **(2) “+” button** to add a new plan directly to the grid and a **(3) “Return” button** that takes users back to the main calendar view in a single tap. These controls remain visible during interaction, providing quick, predictable access to the most common actions. On the left, a scrollable **(4) time grid** lists all 24 hours in military time (00:00–23:00), providing precise times for scheduling. When users add a plan, it appears as a **(5) block** whose height spans from the chosen start time to end time, aligning with the hourly grid, so longer plans occupy more vertical space, making durations and gaps easy to spot at a glance. Tapping a block can pop up details (such as title) and options to edit or remove it, allowing users to manage the day without leaving the page.

Screen Sketch 8 (Ayaan Syed):



The Settings page lets users view their current name, username, email, and profile picture, and make changes as needed. Users can edit these fields by clicking directly on each one; for example, to replace the profile picture or remove items they no longer want displayed (although fields can't be empty). The page also includes an option to permanently delete the account, which requires further confirmation to delete the user from the database.

At the top of the page, the **(1) “User Profile” title** clearly states the page's purpose and anchors the layout. To its right, a **(2) “Return” button** provides a quick way back to the QuestBuddy home screen, keeping primary navigation close at hand.

Centered on the page is a **(3) circular profile picture** that displays the user's current image; selecting it lets users change the picture when desired. Below that, three editable text fields, **(4) Full Name**, **(5) Username**, and **(6) Email**, show the current values and can be tapped to update, keeping review and editing in one place for efficiency.

At the bottom, a (7) **“Delete Account” button** offers the option to permanently remove the account. Selecting this control triggers a confirmation pop-up to verify intent before proceeding, helping prevent accidental deletion.